

Toollery: Self-Verified Query Retrieval for Scalable Tool and Skill Selection

Anonymous ACL submission

Abstract

Large language model (LLM) agents increasingly operate over large and evolving libraries of tools, APIs, and skills. In this setting, tool selection becomes a scalability bottleneck: placing all candidate specifications in the prompt is costly, often exceeds context limits, and can reduce selection accuracy by forcing the model to reason over many irrelevant or semantically adjacent options. We present **Toollery**, a retrieval framework for scalable tool and skill selection. Toollery is built on a simple observation: users describe tasks as intents, while tool and skill metadata describe capabilities; matching intents to other verified intents is often more reliable than matching intents directly to terse metadata. Offline, Toollery generates diverse candidate queries for each tool or skill, then applies round-trip self-verification to retain only queries that reliably lead a verifier model back to the source tool or skill. The retained *verified queries* form a *verified query index*. Online, a user query retrieves similar verified queries from this index, aggregates their associated tools or skills, and constructs a compact candidate subset. Only compact candidate specifications are passed to the final LLM for selection and argument generation; verified queries are used solely for retrieval and are not included in the final LLM prompt. This design decouples large-library search from final decision making, enabling existing LLM agents to scale to larger tool and skill collections without fine-tuning.

1 Introduction

Large language models (LLMs) are increasingly used as agents that call external tools, APIs, and skills to interact with software systems and real-world services (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024; Yao et al., 2022a). In such systems, the model must first decide which tool or skill is relevant to a user request, and only then generate

a valid invocation. This selection step is deceptively difficult. It is not merely a matter of reading a function name or API description: real users express goals in underspecified, contextual, and domain-specific language, while tool and skill specifications are often terse, implementation-oriented, or written for developers rather than end users.

The difficulty becomes more pronounced as the available library grows. A small agent may expose a handful of functions, but practical assistants increasingly need access to hundreds or thousands of tools and skills: operating-system actions, enterprise APIs, cloud services, database operations, retrieval modules, and reusable workflow skills (Wu et al., 2024; Amazon Web Services, 2024; Mo et al., 2025; Xu et al., 2025). The common prompting strategy of placing every candidate specification in context does not scale well. It increases latency and cost, can exceed context windows, and forces the LLM to compare many irrelevant or near-duplicate candidates. Even when the full library fits in context, long candidate lists can distract the model and degrade selection accuracy (Paramanayakam et al., 2025).

Existing retrieval-based approaches mitigate this problem by retrieving tools from their names, descriptions, or documentation before prompting the LLM (Agarwal et al., 2025; Lin et al., 2025). However, direct query-to-metadata retrieval exposes a semantic mismatch. A user may ask, “Can you tell me whether I should bring an umbrella to my afternoon meeting?”, while the relevant tool may be specified as `get_hourly_forecast(location, time)`. The user query and the tool metadata refer to the same capability, but at different levels of abstraction. As the library grows, this mismatch becomes increasingly costly: generic words in metadata can attract false positives, while specialized tools may be missed because their documentation does not resemble natural user intent.

This paper introduces **Toollery**, a framework for

self-verified query retrieval for scalable tool and skill selection. Toollery is based on a core insight: instead of retrieving tools or skills directly from static metadata, we can first construct a verified intent layer around each candidate. Offline, a generator LLM produces diverse *candidate queries* that express plausible user requests for each tool or skill. A verifier LLM then performs a round-trip check: given a generated query and a candidate pool containing the source tool or skill plus distractors, the verifier must select the original source. Only generated queries that pass this check are retained as *verified queries*. The resulting *verified query index* maps natural-language intents to tools and skills.

At inference time, Toollery uses the incoming user query only to retrieve from this verified query index. Retrieved verified queries vote for their associated tools or skills, and Toollery aggregates these signals into a *compact candidate subset*. The final LLM receives the user query together with compact specifications for only this subset, then performs the ordinary final decision and argument generation. Crucially, the verified queries themselves do *not* enter the final LLM prompt. They are retrieval-time artifacts: their role is to find a small, high-recall set of candidates, not to steer or explain the final answer. This separation keeps the final prompt compact and prevents generated text from contaminating the model’s final tool-use reasoning.

Toollery is model-agnostic and does not require fine-tuning. It can be added as an indexing and retrieval layer in front of existing function-calling or agent frameworks. It also naturally covers both *tools*, such as APIs with structured schemas, and *skills*, such as reusable procedures or agent capabilities described in natural language. In both cases, the system faces the same selection problem: map a user intent to the small set of executable or invocable capabilities that the LLM should consider.

Our contributions are:

- **A self-verified query retrieval formulation.** We frame scalable tool and skill selection as retrieval over verified user-intent queries rather than direct retrieval over tool metadata. This reformulation aligns the retrieval space with how users express needs.
- **A round-trip verification pipeline.** We propose an offline procedure that generates candidate queries for each tool or skill and filters them through verifier-based round-trip se-

lection against distractors, producing a high-precision verified query index.

- **A compact candidate selection interface for LLM agents.** At inference time, Toollery retrieves verified queries, aggregates their associated tools or skills, and passes only compact candidate specifications to the final LLM. Verified queries are used only inside the retrieval index and are excluded from the final prompt.

2 Related Work

Tool-augmented LLMs. Tool-augmented language models extend LLMs from text generation to action generation by allowing them to call external APIs, search engines, calculators, software interfaces, and other executable resources. Early work showed that models can learn when and how to invoke tools from demonstrations or self-supervised traces (Schick et al., 2023; Yao et al., 2022b), and subsequent agent benchmarks have emphasized planning, execution, and interaction in realistic environments (Yao et al., 2022a; Huang et al., 2024). A large body of work improves tool use by instruction tuning or supervised fine-tuning on tool-call trajectories. Gorilla (Patil et al., 2024), ToolLLM (Qin et al., 2023), API-Bank (Li et al., 2023), ToolACE (Liu et al., 2024a), and APIGen (Liu et al., 2024b) demonstrate that tool-specific training can improve schema following, argument generation, and function-call formatting. These approaches are complementary to Toollery: they improve the base model’s ability to execute a chosen tool call, while Toollery focuses on reducing the candidate set that the model must inspect before making that call. This distinction is important in large tool libraries, where even a strong function-calling model may fail because the relevant schema is buried among hundreds or thousands of distractors.

Tool and skill retrieval and routing. A second line of work treats tool selection as retrieval or routing. Rather than presenting every available tool to the LLM, the system first retrieves a smaller set of candidate tools and then asks the LLM to choose among them. ToolRM (Agarwal et al., 2025), MassTool (Lin et al., 2025), RAG-MCP (Gan and Sun, 2025), and context-pruning approaches such as Less is More (Paramanayakam et al., 2025) share this broad motivation: tool use improves when the model reasons over a compact and relevant action

space instead of a long, noisy tool prompt. SkillRouter is especially related because it also aims to route user requests to a subset of skills before final model inference. However, SkillRouter performs compact full-text retrieval and reranking over the skill body or skill documentation. Toolllery makes a different design choice. It does not retrieve directly over the full tool description as the primary semantic object; instead, it constructs a self-verified generated-query index offline and performs query-level retrieval at inference time. In this sense, Toolllery is not a universal skill router and is not intended to replace SkillRouter. It can be viewed as a pre-routing candidate compression layer that supplies a smaller, semantically grounded candidate set to an existing router or to a standard function-calling LLM.

Query augmentation and verification. Toolllery is also connected to query augmentation methods that generate alternative user intents, pseudo-queries, or synthetic supervision in order to improve retrieval recall. In document retrieval, generated queries can make sparse or dense indexes more robust to vocabulary mismatch; in tool selection, the analogous mismatch is between user intent and tool metadata. A tool description usually states what an API does, while a user request states the goal they want to accomplish. Directly embedding the tool description and the user request therefore asks the retriever to bridge a substantial semantic gap. Toolllery addresses this gap by generating diverse natural-language queries for each tool and then filtering them through round-trip self-verification. The verification step is central: a generated query is retained only when an independent verifier can recover the originating tool from a candidate set containing distractors. The resulting verified queries are not merely paraphrases of the tool description; they are high-precision semantic anchors that have survived an explicit ambiguity check. This makes the index better suited for candidate compression, where false positives can crowd out the correct tool under a fixed top- k budget.

RAG baselines and generic document retrieval. Retrieval-augmented generation systems retrieve external textual evidence and condition the LLM on the retrieved documents. Generic RAG frameworks, including systems such as RAGAnything, are designed for broad document grounding across heterogeneous corpora and modalities. Toolllery differs in both the indexed unit and the downstream

contract. A generic RAG baseline retrieves chunks of documentation and sends them to the LLM as evidence. Toolllery indexes verified queries, uses them only to identify candidate tools, and sends the LLM compact tool specifications rather than arbitrary retrieved text. This distinction matters because the final task is not open-ended answer generation but constrained tool selection and argument generation. The retrieval layer should therefore optimize for selection quality and context compression, not for document coverage. Toolllery can be implemented using standard dense retrieval infrastructure, but its contribution lies in the construction and use of a verified query index specialized for tool candidate selection.

3 Method

Toolllery separates large-library search from final LLM decision making. The offline stage builds a verified query index for a library of tools and skills. The online stage retrieves from this index to construct a compact candidate subset, then asks the LLM to select and invoke from that subset. This section describes the problem setup and the first half of the offline pipeline: query generation and round-trip self-verification.

3.1 Problem Setup

Let $\mathcal{A} = \{a_1, \dots, a_M\}$ denote a library of available actions, where each action a_i may be a structured tool, an API function, or a higher-level skill. Each action is associated with a specification s_i , which may include a name, natural-language description, argument schema, usage constraints, examples, or skill instructions. Given a user query x , the goal is to identify a small candidate set $\mathcal{C}(x) \subset \mathcal{A}$ that contains the action(s) needed to satisfy the request. A downstream LLM then receives x and compact specifications for $\mathcal{C}(x)$, and outputs the final selected action and arguments.

The key constraint is that M can be large. Passing all specifications $\{s_i\}_{i=1}^M$ to the LLM is expensive and may be infeasible. More importantly, it gives the LLM a noisy comparison problem: it must attend to many candidates that are irrelevant to the current user query. We therefore seek a retrieval function

$$R(x, \mathcal{I}) \rightarrow \mathcal{C}(x), \quad (1)$$

where $\mathcal{I}$ is an index built offline from the action library, and $|\mathcal{C}(x)| \ll M$. The retrieval function

should have high recall for the correct action while keeping the candidate subset compact enough for reliable LLM selection.

Toollery builds $\mathcal{I}$ not from action metadata alone, but from verified natural-language queries associated with each action. For each action a_i , the offline pipeline constructs a set of verified queries $V_i = \{v_{i1}, \dots, v_{in_i}\}$. The full verified query index is

$$\mathcal{I} = \{(v, a_i) : v \in V_i, a_i \in \mathcal{A}\}. \quad (2)$$

Each entry links a natural-language intent v to the action a_i that can satisfy it. During online retrieval, user query x is compared against the verified queries in $\mathcal{I}$, not against the final action specifications directly.

We distinguish three textual objects throughout the method. *Candidate queries* are generated offline from an action specification and may contain noise. *Verified queries* are candidate queries that pass round-trip self-verification. *Compact candidate specifications* are shortened action descriptions passed to the final LLM after retrieval. Verified queries belong only to the retrieval index; they are not shown to the final LLM.

3.2 Query Generation for Tools and Skills

The first offline step expands each action specification into diverse candidate queries. For an action a_i with specification s_i , a generator model G produces a set

$$Q_i = G(s_i; N, \pi), \quad (3)$$

where N is the target number of generated queries and π is a generation prompt that instructs the model to write plausible user requests that would require a_i .

The purpose of this step is not to create additional documentation for the final LLM. Instead, the generated queries act as retrieval anchors that translate tool- or skill-centric descriptions into user-centric language. For structured tools, the generator is given the tool name, description, argument schema, return type when available, and any examples. For skills, the generator is given the skill name, capability description, triggering conditions, expected inputs, and operational constraints. In both cases, the generator is asked to produce queries that a real user might write before knowing the action exists.

We encourage diversity along several dimensions:

- **Lexical and syntactic variation.** Queries should use different phrasings rather than repeating the words in the specification.
- **Intent abstraction.** Queries should express user goals, not API names or implementation details. For example, a weather API may correspond to a travel-planning question rather than an explicit request to call a forecast endpoint.
- **Contextual specificity.** Queries may include concrete entities, constraints, or situational context, as long as the source action remains sufficient to satisfy the request.
- **Boundary coverage.** Queries should cover important use cases and input regimes described by the action specification, including optional arguments and common constraints.

Generated candidate queries are intentionally over-inclusive. Some will be too broad, too narrow, ambiguous across multiple actions, or subtly inconsistent with the source specification. For instance, a generated query for a calendar lookup skill might accidentally require editing an event, or a generated query for a file search tool might imply semantic search when the tool only supports exact filename matching. Toollery therefore treats Q_i as a noisy proposal set and applies verification before indexing.

3.3 Round-Trip Self-Verification

Round-trip self-verification filters generated candidate queries by testing whether they lead a verifier model back to the source action. For each generated query $q \in Q_i$, Toollery constructs a verification pool

$$\mathcal{P}_{i,q} = \{a_i\} \cup \mathcal{D}_{i,q}, \quad (4)$$

where $\mathcal{D}_{i,q} \subset \mathcal{A} \setminus \{a_i\}$ is a set of distractor actions. The verifier model H receives the generated query q and compact specifications for the actions in $\mathcal{P}_{i,q}$, then selects the action that best satisfies the query or abstains if none is appropriate:

$$\hat{a} = H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}). \quad (5)$$

The query is retained only if $\hat{a} = a_i$. Otherwise, it is discarded. The verified query set for action a_i is therefore

$$V_i = \{q \in Q_i : H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}) = a_i\}. \quad (6)$$

Algorithm 1 Offline Construction of the Verified Query Index

```

1: Input: action library  $\mathcal{A}$ ; specifications  $\{s_i\}$ ;
   generator  $G$ ; verifier  $H$ ; query count  $N$ ; dis-
   tractor sampler SampleDistractors
2: Output: verified query index  $\mathcal{I}$ 
3:  $\mathcal{I} \leftarrow \emptyset$ 
4: for each action  $a_i \in \mathcal{A}$  do
5:    $Q_i \leftarrow G(s_i; N)$   $\triangleright$  Generate candidate
     queries
6:   for each candidate query  $q \in Q_i$  do
7:      $\mathcal{D}_{i,q} \leftarrow \text{SampleDistractors}(a_i, q, \mathcal{A})$ 
8:      $\mathcal{P}_{i,q} \leftarrow \{a_i\} \cup \mathcal{D}_{i,q}$ 
9:      $\hat{a} \leftarrow H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\})$ 
10:    if  $\hat{a} = a_i$  then
11:      Add  $(q, a_i)$  to  $\mathcal{I}$   $\triangleright q$  becomes a
        verified query
12:    end if
13:  end for
14: end for
15: return  $\mathcal{I}$ 

```

This check is a round trip: generation maps an action specification to a possible user query, and verification maps that query back to an action under competition. A generated query that cannot recover its source action is not a reliable retrieval anchor. It may be linguistically plausible, but it is unsafe to index because it could retrieve the wrong action when the library grows.

Distractor selection is important. Random distractors test whether a query is broadly compatible with unrelated actions, while hard distractors test whether the query distinguishes among semantically adjacent tools or skills. In practice, Toollery can combine both: sample random actions for coverage and retrieve metadata-nearest neighbors as hard negatives. The verifier is thus asked to resolve realistic confusions, such as read versus write actions, search versus summarization skills, or APIs that differ only in argument constraints.

The output of self-verification is the verified query index $\mathcal{I}$. Each index entry stores the verified query text, its embedding or sparse representation, and the identifier of its source action. It may also store lightweight metadata such as the source category or verification confidence. The full action specification remains outside the verified query index and is retrieved only after action aggregation, when Toollery constructs the compact candidate subset for the final LLM.

Because verified queries are produced offline, the cost of generation and verification is amortized across future user requests. Updating the library is also local: when a new tool or skill is added, Toollery generates and verifies queries for that action and inserts the passing entries into the verified query index. When an action is modified or removed, its associated verified queries can be re-generated or deleted without rebuilding the entire system.

After the verified query index has been constructed, online inference proceeds by embedding or otherwise representing the incoming user query, retrieving the nearest verified queries, aggregating their source actions, and forming a compact candidate subset. The final LLM sees only the user query and compact candidate specifications for this subset. The verified queries remain internal to retrieval, preserving a clean separation between scalable search and final LLM-based tool or skill selection.

3.4 Verified Query Retrieval

After the offline construction stage, Toollery has an index $\mathcal{I}$ containing verified query–tool pairs. Each entry is a tuple (q_i, t_i, s_i) , where q_i is a generated user-like query, $t_i \in \mathcal{T}$ is the tool that the query was generated for, and s_i optionally stores the verification signal, such as the verifier’s confidence or the rank of the target tool during verification. The key property of $\mathcal{I}$ is that each query has been tested for functional alignment: when the verifier receives q_i together with the target tool and sampled distractors, it must select t_i . Toollery therefore retrieves over a space of verified intents rather than over raw tool descriptions.

At inference time, given a user request x , Toollery embeds x and the verified queries into a shared dense space. Let $e(x)$ denote the embedding of the user request and $e(q_i)$ denote the embedding of a verified query. The retrieval score for each verified query is

$$r_i = \cos(e(x), e(q_i)). \quad (7)$$

Toollery then retrieves the top- m verified queries by r_i . This stage is intentionally lightweight: it can be implemented with any standard vector index and does not require updating the backbone LLM. The important difference from direct tool retrieval is the semantic target. Direct retrieval asks whether a user request resembles a tool description, which

is often written in API-centric language. Verified query retrieval asks whether a user request resembles another user-like intent that has already been validated as mapping to a tool.

Because multiple verified queries may point to the same tool, Toollery aggregates query-level evidence into tool-level scores. For a tool t , let $\mathcal{N}_m(t)$ be the set of retrieved verified queries among the top- m whose target tool is t . A simple aggregation function is

$$R(t | x) = \max_{q_i \in \mathcal{N}_m(t)} r_i, \quad (8)$$

which rewards the best-matching verified intent for each tool. In practice, other aggregators can be used, such as the mean of the top few scores, a log-sum-exp score, or a weighted score that incorporates verification confidence s_i . We use aggregation to convert many query-level matches into a stable ranked list of tools while avoiding repeated copies of the same tool in the final context.

3.5 Candidate Subset Construction and LLM Selection

The output of verified query retrieval is not the final tool call. Instead, it is a compressed candidate subset passed to the LLM for final selection and argument generation. Toollery ranks tools by $R(t | x)$ and selects the top- k unique tools:

$$\mathcal{C}_k(x) = \text{TopK}_{t \in \mathcal{T}} R(t | x). \quad (9)$$

For each selected tool, Toollery instantiates the compact tool specification required by the target function-calling interface, including the tool name, natural-language description, and parameter schema. The LLM then receives the original user request x together with $\mathcal{C}_k(x)$ and produces the final tool call.

This two-step design preserves the role of the LLM as the final decision-maker. Toollery does not assume that nearest-neighbor retrieval alone is sufficient for tool use, especially when tools have overlapping descriptions or when the user request requires careful argument extraction. Instead, retrieval narrows the action space and the LLM resolves the remaining ambiguity. This makes Toollery compatible with native function-calling APIs, prompt-based tool calling, and external routers. When used with SkillRouter-like systems, Toollery can operate before the router as a candidate compression layer; when used without

an additional router, the compressed subset can be sent directly to the backbone LLM.

The top- k value controls the recall-compression trade-off. A larger k increases the probability that the ground-truth tool is included, but also increases prompt length and distractor pressure. A smaller k improves efficiency and reduces irrelevant schemas, but can hurt accuracy if retrieval misses the correct tool. The central empirical question is therefore whether verified query retrieval can maintain high candidate recall with small k as the global tool pool grows. Our proposed experiments evaluate this question under controlled candidate-set expansion.

3.6 Token and Latency Analysis

Toollery is motivated not only by selection quality but also by the token and latency costs of uncompressed tool prompting. Let N be the number of tools in the full library, and let L_t be the average token length of a tool specification. A standard full-prompting approach sends approximately

$$O(N \cdot L_t) \quad (10)$$

tool-definition tokens to the LLM for each request, in addition to the user prompt and system instructions. As N grows from tens to hundreds or thousands, the prompt can exceed the model’s context window or consume most of the available budget before the model has reasoned about the user request.

Toollery changes the online LLM context from full-library prompting to top- k candidate prompting. The LLM receives approximately

$$O(k \cdot L_t) \quad (11)$$

tool-definition tokens, where $k \ll N$ and is fixed independently of the full library size. The retrieval index still grows with the tool library, but dense vector search is substantially cheaper than long-context LLM inference and can be served using standard approximate nearest-neighbor infrastructure. The offline cost of generating and verifying queries is amortized over many future requests and only needs to be rerun for new or modified tools.

This distinction is important for latency. In full-prompting systems, both prefill cost and attention cost increase with the number of candidate tools sent to the model. In Toollery, the LLM-facing prompt length remains nearly constant once k is fixed. The online pipeline adds an embedding and retrieval step, but this step replaces a much larger

LLM context expansion. Toollery should therefore be evaluated as a selection-quality and compression mechanism: it aims to preserve or improve final tool-call accuracy while reducing the number of tool tokens and the amount of irrelevant schema information presented to the LLM.

4 Proposed Experiments

This section specifies the evaluation plan for Toollery as a self-verified query retrieval framework for skill and atomic-skill selection. We use *skill* as the general interface unit and *atomic skill* for the original tool-style APIs whose behavior can be described by a name, natural-language description, schema, and optional manual. The central question is not whether a larger prompt can contain more generated text, but whether self-verified usage queries can build a better retrieval index while keeping the final LLM decision prompt compact. Unless otherwise noted, generated and verified queries are used only inside the retrieval index. The final LLM selector receives only the top- k compact candidate skill specifications, not the generated query pool.

4.1 Research Questions

RQ1: Selection accuracy. Does Toollery improve skill selection over raw full-text retrieval, dense retrieval, reranking, SkillRouter-style pipelines, and RAG-style baselines?

RQ2: Benefit of query augmentation. How much of the gain comes from adding generated usage queries to the index, and how much comes specifically from Toollery’s round-trip self-verification?

RQ3: Skill and atomic-skill generality. Does the same retrieval and verification design work on both SkillBench-style skill routing tasks and the original atomic tool benchmarks such as HammerBench, xLAM, BFCL, and ScaleTool?

RQ4: Efficiency under large candidate pools. Can Toollery reduce final-prompt tokens, LLM selection latency, and monetary cost by retrieving a compact subset before the LLM sees the candidates, while preserving or improving selection quality?

RQ5: Robustness to distractors and scale. How does performance change as the candidate skill pool grows from small benchmark pools to large mixed pools with many semantically similar distractors?

4.2 Benchmarks

SkillBench / SkillRouter-compatible track.

This track evaluates Toollery on skill routing datasets where each instance contains a user request and one or more gold skills. To make the comparison compatible with SkillRouter-style evaluation, we report retrieval-oriented metrics such as Hit@ k , Recall@ k , MRR, and nDCG whenever the annotations support ranked relevance. When executable task environments or validated final answers are available, we additionally report final task success after the LLM selects from the retrieved candidate subset.

Atomic tool/skill track. This track keeps the original Toollery experiments on HammerBench, xLAM, BFCL, and ScaleTool, but reframes each tool as an atomic skill. Each atomic skill is represented by its original metadata, including name, description, schema, manual, and example fields when available. We retain the original evaluation metrics from the paper for comparability, and add retrieval metrics plus efficiency measurements to isolate the effect of candidate pruning.

Large-pool and distractor settings. For both tracks, we create controlled scale settings by increasing the number of candidate skills available at inference time. Pool sizes should include the native benchmark pool and larger mixed pools, for example $N \in \{100, 500, 1,000, 5,000, 10,000\}$ when data availability allows. We also create hard-distractor pools by mixing skills with overlapping names, descriptions, schemas, or task domains. This setting tests whether verified queries improve grounding beyond surface-level name and description matching.

4.3 Baselines and Settings

We separate method families from information settings. This avoids treating raw metadata retrieval as if it were a Toollery variant. Toollery is defined by query-level retrieval plus self-verification; raw metadata retrieval is a baseline setting, not a Toollery variant.

Information settings. **Raw Baselines** use only the original skill metadata, such as name, description, schema, manual, body, and examples. **Query-Augmented Baselines** add the same generated usage queries to each baseline’s searchable index to test whether query generation alone improves retrieval. **Unverified Toollery** retrieves over gen-

erated usage queries and aggregates query-level matches to skill-level candidates, but does not apply round-trip self-verification. **Full Toollery** generates candidate usage queries, filters them through round-trip self-verification, indexes only verified queries, retrieves matching verified queries at inference time, aggregates them into a top- k skill subset, and sends only compact candidate skill specifications to the final LLM selector.

Retrieval baselines. We include BM25 over raw skill text, dense retrieval over raw skill text, and a retrieve-and-rerank pipeline that mirrors the SkillRouter family when model access and training data are available. For the SkillBench-compatible track, the SkillRouter comparison should use the official encoder and reranker setup where possible. If reproducing the full training recipe is infeasible, we report a frozen off-the-shelf dense retriever plus reranker and label it as an adapted SkillRouter-style baseline rather than the official SkillRouter model.

RAGAnything baseline. RAGAnything is adapted as a generic retrieval-over-documentation baseline. For raw settings, it indexes each skill’s original documentation fields. For query-augmented settings, it receives the same generated queries used by the other baselines. This comparison tests whether Toollery’s gain comes from generic document retrieval or from the verified query-to-skill retrieval structure.

LLM selector baselines. For small pools, we include a direct LLM selector that sees all candidate skill specifications. For larger pools, the all-candidate prompt may exceed context or be economically unrealistic; in that case we report the maximum feasible pool size and compare against retrieval-preselected LLM selection. The final selector model, prompt template, temperature, maximum output length, and candidate format must be fixed across methods.

4.4 Metrics

Retrieval and routing metrics. For single-gold tasks, we report Hit@ k for $k \in \{1, 3, 5, 10, 20\}$ and MRR. For multi-gold tasks, we report Recall@ k and nDCG@ k when graded or multi-relevance labels are available. We also report candidate-set recall before the final LLM selection step, because Toollery’s first-stage objective is to retain the gold skill inside a compact subset.

Final task metrics. On executable or answer-validated benchmarks, we report final task success, exact match, pass rate, or the benchmark’s official score. The key distinction is between retrieval success and end-to-end success: a method can retrieve the gold skill but fail if the final LLM chooses incorrectly or formats the call incorrectly.

Efficiency metrics. We measure the number of candidates sent to the final LLM, final prompt tokens, final completion tokens, retrieval latency, reranking latency, final LLM selection latency, total inference latency, and estimated cost per 1,000 user queries. We also report compression ratios relative to the direct all-candidate LLM selector:

$$\text{TokenCompression} = \frac{\text{PromptTokens}_{\text{all}}}{\text{PromptTokens}_{\text{method}}}, \quad (12)$$

$$\text{LatencyCompression} = \frac{\text{LLMLatency}_{\text{all}}}{\text{LLMLatency}_{\text{method}}}. \quad (13)$$

When all-candidate prompting is infeasible for large pools, we use the largest feasible direct-LLM pool as an upper-bound reference and clearly mark larger pool comparisons as retrieval-only or retrieval-plus-LLM-subset evaluation.

Verification metrics. For the offline query construction stage, we report the number of generated queries per skill, the self-verification acceptance rate, the average number of verified queries per skill, the percentage of skills with zero verified queries, offline construction cost, and offline construction latency. These metrics separate one-time indexing cost from per-query serving cost.

4.5 Ablations

Generated queries without verification. We compare Raw Baselines, Query-Augmented Baselines, Unverified Toollery, and Full Toollery. This is the main ablation requested by the current paper positioning. If generated queries help every method, Query-Augmented Baselines should improve over Raw Baselines. If round-trip self-verification removes noisy or misleading generated queries, Full Toollery should improve over Unverified Toollery, especially in hard-distractor settings.

Verification strictness. We vary the round-trip self-verification threshold or acceptance rule. Candidate queries can be accepted if the verifier recovers the source skill at top-1, top-3, or top-5, or if the

verifier assigns a score above a calibrated threshold. This ablation traces the tradeoff between coverage and precision in the verified query index.

Number of generated queries. We vary the number of generated candidate queries per skill before verification, for example $m \in \{5, 10, 20, 40\}$. The expected analysis is whether larger query pools improve recall until verification or retrieval becomes saturated.

Candidate subset size. We vary the number of compact candidates sent to the final LLM, for example $k \in \{1, 3, 5, 10, 20\}$. This ablation directly measures the quality-cost frontier: smaller k should reduce tokens and LLM latency, while larger k should increase candidate-set recall and potentially final task success.

Index field ablation. We compare indexes over name-only, name plus description, full raw text, raw text plus generated queries, generated queries only, and verified queries only. This helps position Toolery against SkillRouter-style full-text retrieval and tests whether verified query text captures usage semantics that raw documentation misses.

Aggregation ablation. Because Toolery retrieves at the query level and then aggregates to the skill level, we compare aggregation rules such as max score, mean top- r score, sum of top- r scores, reciprocal-rank fusion, and learned lightweight aggregation if training data are available.

4.6 Efficiency, Token, and Latency Analysis

The efficiency claim should be evaluated as a first-class result, not a side note. Toolery may build a larger offline retrieval index, but that index is not included in the final LLM prompt. At serving time, the system retrieves verified queries, maps them back to a small skill subset, and sends only compact candidate specifications to the final LLM selector. Therefore the online token cost scales with k , not with the full number of indexed queries or the full skill pool.

For each method and pool size, we log:

- **Index size:** total indexed documents, average indexed entries per skill, and average indexed tokens per skill.
- **Final prompt size:** number of candidates sent to the LLM and prompt tokens after candidate pruning.

- **Latency:** retrieval latency, reranking latency, final LLM selection latency, and total latency.

- **Cost:** estimated per-query LLM cost and total cost per 1,000 queries, separating offline indexing cost from online serving cost.

- **Quality-efficiency frontier:** final success or Hit@ k versus prompt tokens and total latency.

The main efficiency comparison should include three practically important systems: direct all-candidate LLM selection, SkillRouter/RAG-style retrieval plus LLM selection over a subset, and Full Toolery retrieval plus LLM selection over a subset. This distinguishes prompt compression from retrieval quality.

4.7 Expected Tables

4.8 Experiments to Run

1. Build the SkillBench / SkillRouter-compatible evaluation pipeline: normalize skill metadata, define gold labels, implement Hit@ k , Recall@ k , MRR, nDCG, and final task success when supported.
2. Re-run the original Toolery benchmark suite on HammerBench, xLAM, BFCL, and Scale-Tool with terminology changed from tool to atomic skill.
3. Implement Raw Baselines for BM25, dense retrieval, reranker retrieval, RAGAnything, and direct LLM selection where context length permits.
4. Generate the shared query pool for every skill. Store the generated queries once and reuse the identical pool for Query-Augmented Baselines, Unverified Toolery, and Full Toolery.
5. Run Query-Augmented Baselines by adding the shared generated queries to each baseline’s searchable index without using Toolery’s round-trip verification filter.
6. Run Unverified Toolery with query-level retrieval and skill-level aggregation, using generated queries but no self-verification.
7. Run Full Toolery with round-trip self-verification, a verified query index, query-level retrieval, skill-level aggregation, and compact top- k final LLM selection.

Setting	Raw fields	Generated queries	Self-verification	Retrieval unit	Final LLM input	Purpose
Raw Baselines	✓	–	–	Skill document	Top- k compact specs	Raw metadata retrieval
Query-Augmented Baselines	✓	✓	–	Skill document or query-augmented document	Top- k compact specs	Effect of generation alone
Unverified Toollery	Optional	✓	–	Generated query	Top- k compact specs	Query-level retrieval without filtering
Full Toollery	Optional	✓	✓	Verified query	Top- k compact specs	Verified query index plus compact LLM selection

Table 1: Experimental settings. Generated or verified queries are indexed for retrieval but are not copied into the final LLM prompt. The final selector sees only compact specifications for the retrieved top- k skills.

Track	Method	Hit@1	Hit@5	Recall@10	MRR	nDCG@10	Final Success
SkillBench	BM25 (Raw)	–	–	–	–	–	–
SkillBench	Dense Retriever (Raw)	–	–	–	–	–	–
SkillBench	SkillRouter / SkillRouter-style	–	–	–	–	–	–
SkillBench	RAGAnything (Raw)	–	–	–	–	–	–
SkillBench	RAGAnything (+ generated queries)	–	–	–	–	–	–
SkillBench	Unverified Toollery	–	–	–	–	–	–
SkillBench	Full Toollery	–	–	–	–	–	–
Atomic skills	BM25 (Raw)	–	–	–	–	–	–
Atomic skills	Dense Retriever (Raw)	–	–	–	–	–	–
Atomic skills	RAGAnything (Raw)	–	–	–	–	–	–
Atomic skills	RAGAnything (+ generated queries)	–	–	–	–	–	–
Atomic skills	Unverified Toollery	–	–	–	–	–	–
Atomic skills	Full Toollery	–	–	–	–	–	–

Table 2: Main selection and end-to-end results. Use SkillBench-compatible metrics on the skill-routing track and the original benchmark official score for the atomic-skill track when available. All entries are placeholders until the experiments are run.

8. Add RAGAnything on the original atomic-skill benchmarks in both raw and query-augmented forms.
9. If feasible, evaluate official SkillRouter on SkillBench. If not feasible, clearly label the reproduced comparison as SkillRouter-style and document the encoder, reranker, and training or zero-shot setup.
10. Measure token count, retrieval latency, reranking latency, final LLM latency, total latency, and estimated cost for each method and pool size.
11. Run ablations over verification strictness, number of generated queries per skill, candidate subset size k , index fields, and aggregation rule.
12. Run scale and hard-distractor evaluations by increasing candidate pool size and adding semantically similar skills.

4.9 Implementation Notes

Fair query augmentation. The generated query pool must be identical across Query-Augmented Baselines, Unverified Toollery, and Full Toollery before verification. This controls for the benefit of generation itself. Full Toollery may remove queries through self-verification, but it should not receive additional private generated queries that are unavailable to the ablations.

No generated queries in final prompts. For all Toollery variants and query-augmented baselines, final LLM prompts should contain only compact candidate skill specifications. If a baseline uses generated queries in the final prompt, report it as a separate prompt-expansion baseline because it changes the token budget and weakens the compression claim.

Prompt and model control. Use the same final selector model, system prompt, candidate formatting, temperature, and decoding parameters for all methods. The only allowed difference should be the retrieved candidate subset and its order.

Latency measurement. Measure retrieval, reranking, and final LLM latency separately. Report mean, median, and p95 if possible. Offline generation and verification costs should be reported separately from online serving latency.

Index construction. Store raw skill documents, generated queries, and verified queries as separate index fields or separate indexes. This makes it possible to run field ablations without regenerating data.

RAGAnything adaptation. Document exactly how each skill is serialized into RAGAnything documents and how retrieved chunks are mapped back to skill IDs. If RAGAnything retrieves chunks rather than skills, aggregate chunk scores to skill scores before top- k final LLM selection.

Method / Ablation	Gen. queries / skill	Accepted / skill	Accept rate	Zero-query skills	Hit@5	Final Success	Offline Cost
Raw Baseline	0	0	—	—	—	—	—
Query-Augmented Baseline	m	m	100%	0%	—	—	—
Unverified Toollery	m	m	100%	0%	—	—	—
Full Toollery (top-1 verification)	m	—	—	—	—	—	—
Full Toollery (top-3 verification)	m	—	—	—	—	—	—
Full Toollery (score threshold)	m	—	—	—	—	—	—

Table 3: Verification ablation. This table isolates the contribution of round-trip self-verification from query generation itself. Offline cost should be reported separately from online inference cost.

Method	Pool size	Cand. to LLM	Prompt toks	Tok. comp.	Retr. ms	Rerank ms	LLM ms	Cost / 1k
Direct all-candidate LLM	—	—	—	1.0×	—	—	—	—
BM25 + LLM subset	—	k	—	—	—	—	—	—
SkillRouter-style + LLM subset	—	k	—	—	—	—	—	—
RAGAnything + LLM subset	—	k	—	—	—	—	—	—
Unverified Toollery + LLM subset	—	k	—	—	—	—	—	—
Full Toollery + LLM subset	—	k	—	—	—	—	—	—

Table 4: Token, latency, and cost compression. Generated and verified queries increase the retrieval index size but do not enter the final LLM prompt. The online LLM prompt contains only the compact specifications of the retrieved top- k candidates.

Statistical reporting. Run at least three seeds for query generation and LLM selection when budget allows. Report confidence intervals or bootstrap standard errors for main results. If generation is expensive, fix one generated query pool and vary only retrieval and final-selection seeds, while explicitly stating this limitation.

Failure analysis. Sample failures where Full Toollery loses to Query-Augmented Baselines or raw retrieval. Categorize them into missing generated coverage, over-strict verification, ambiguous gold labels, schema-level mismatch, and final LLM selection error. This analysis will help explain whether verification improves precision at the cost of recall.

5 Discussion

The central claim of Toollery is deliberately narrow. Toollery is not a universal skill router, a replacement for trained function-calling models, or a general-purpose RAG system over tool documentation. It is a self-verified generated-query retrieval framework for compressing the candidate tool set before LLM selection. This framing clarifies both where Toollery helps and where it should be combined with other components.

The proposed evaluation is designed to test two main observations. First, tool selection degrades as the candidate set grows, even when the ground-truth tool and user request remain unchanged. This suggests that large tool libraries introduce a search-space problem in addition to the familiar challenges of reasoning and argument generation. Second, verified query retrieval can reduce this search-space

pressure by presenting the LLM with a compact set of semantically plausible tools. The improvement comes from changing what the model is asked to do: instead of scanning a large library of heterogeneous schemas, the model performs final selection within a small candidate subset.

This design also explains why Toollery can help both small and large models. Smaller models often have limited context processing ability and are more vulnerable to long prompts containing many irrelevant schemas. Larger models may tolerate longer contexts, but they still pay increased token and latency costs and can be distracted by semantically similar tools. Toollery reduces both types of burden. It compresses the prompt while preserving the final LLM step needed for schema-aware selection and argument filling.

The comparison with SkillRouter and RAGAnything highlights the intended scope. SkillRouter retrieves and reranks over skill bodies; Toollery retrieves over verified generated queries that are used only to produce a candidate set. RAGAnything and related generic RAG systems retrieve document content for grounding; Toollery retrieves intent anchors and ultimately sends compact tool specifications to the model. These systems can be complementary. A deployment could first use Toollery to reduce a large tool library to a manageable subset, then apply a router, reranker, or function-calling model within that subset.

6 Limitations

Toollery depends on the quality of the verified query index. If the teacher model fails to generate queries that cover realistic user intents, retrieval

Method	$N = 100$	$N = 500$	$N = 1,000$	$N = 5,000$	$N = 10,000$	Avg. prompt toks
BM25 (Raw)	–	–	–	–	–	–
Dense Retriever (Raw)	–	–	–	–	–	–
RAGAnything (+ generated queries)	–	–	–	–	–	–
SkillRouter-style	–	–	–	–	–	–
Unverified Toollery	–	–	–	–	–	–
Full Toollery	–	–	–	–	–	–

Table 5: Scaling curve under increasing candidate-pool size. Each cell can report Hit@5, Recall@10, or final success depending on the benchmark. A companion figure should plot quality versus pool size and quality versus final prompt tokens.

recall will be limited no matter how strong the final LLM is. Conversely, if verification is too weak or the distractor set is not challenging enough, ambiguous verified queries may enter the index and produce false-positive candidates. The verification procedure therefore improves precision but does not provide a formal guarantee of correctness.

The current framework also introduces offline maintenance cost. New tools require query generation, verification, embedding, and index updates. This cost is amortized across future calls, but it matters for rapidly changing tool ecosystems. Incremental index construction and targeted re-verification are natural extensions for production settings where tools are frequently added, removed, or modified.

Another limitation is that Toollery primarily addresses single-step candidate selection. It does not by itself solve multi-tool planning, tool sequencing, execution monitoring, or recovery from failed calls. In multi-step agents, Toollery can be applied at each decision point, but the broader planning policy remains outside the scope of the method. Similarly, Toollery compresses the candidate set but does not guarantee correct argument values; the final LLM must still understand the selected schema and fill parameters accurately.

Finally, Toollery inherits the limitations of embedding-based retrieval. Dense retrieval can miss rare intents, domain-specific terminology, or requests that require compositional reasoning across multiple tools. Hybrid retrieval, stronger query aggregation, learned reranking, or domain-adaptive encoders may further improve candidate recall, but these additions should be evaluated against their added complexity and latency.

7 Conclusion

We presented Toollery, a self-verified generated-query retrieval framework for scalable tool selection. The key idea is to move part of the semantic grounding work offline: for each tool, Toollery

generates diverse user-like queries and retains only those that pass round-trip verification against distractor tools. At inference time, the user request is matched against this verified query index, producing a compact candidate subset that is then passed to the LLM for final tool selection and argument generation.

This framing positions Toollery as a pre-routing candidate compression layer rather than a universal router or a generic RAG system. Its goal is to improve selection quality while reducing token and latency costs under large tool pools. By retrieving over verified intents and sending only top- k compact tool specifications to the LLM, Toollery reduces the amount of irrelevant schema information in the prompt and helps existing function-calling pipelines operate more reliably as tool libraries grow.

References

- Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.
- Amazon Web Services. 2024. Boto3: The AWS SDK for Python. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>. Accessed: 2024-05-01.
- Tiantian Gan and Qiyao Sun. 2025. Rag-mcp: Mitigating prompt bloat in llm tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*.
- Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, and 1 others. 2024. Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios. *arXiv preprint arXiv:2401.17167*.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. *API-bank: A comprehensive benchmark for tool-augmented LLMs*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore. Association for Computational Linguistics.
- Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, and 1 others. 2024a. Toolace: Winning the points of llm function calling. *arXiv preprint arXiv:2409.00920*.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, and 1 others. 2024b. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*.
- Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and Zhijie Deng. 2025. Tps-bench: Evaluating ai agents’ tool planning & scheduling abilities in compounding tasks. *arXiv preprint arXiv:2511.01527*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022a. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022b. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.